CSCI 2406: Topic 04

Integer Arithmetic & Performance Basics

Fixed-width arithmetic, condition flags, shifts, and CPU execution-time analysis

Computer Organization & Architecture | AArch64 Reference

Topic 4 Outline

Topic Objective

Understand how fixed-width integer arithmetic works at the machine level, how condition flags support later control flow, and how instruction count, CPI, and clock rate determine execution time.

- **Section 1**: Fixed-width addition, subtraction, and interpretation
- **Section 2**: Condition flags and AArch64 comparison behavior
- **Section 3**: Multiplication, division, and shifts
- **Section 4**: Clock rate, CPI, and execution time
- **Section 5**: Summary and self-test

Section 01

Fixed-Width Integer Arithmetic

Why Integer Arithmetic Matters

Integer Operations Are Everywhere

Integer arithmetic is not limited to explicit calculator-like expressions. It appears constantly in address calculation, loop control, array indexing, comparisons, and resource accounting.

Program-Level Examples

- Increment loop counters.
- Compute array offsets.
- Compare bounds and indices.
- Update pointer values.

Hardware-Level Consequence

- Arithmetic uses fixed-width registers.
- Results can wrap around.
- Flags record selected facts about results.
- Interpretation is supplied by software.

Arithmetic Operates on Bit Patterns

Core Principle

The ALU operates on finite-width bit patterns. Signedness is not a property of the adder itself; it is a property of how software interprets the bits and which branch conditions it later uses.

Same Instruction

```
add x0, x1, x2
```

- Adds two 64-bit values.
- Produces one 64-bit result.
- Does not by itself say signed or unsigned.

Different Readings

- Unsigned: magnitude from 0 to $2^{64} - 1$.
- Signed: two's-complement range from -2^{63} to $2^{63} - 1$.
- Flags help evaluate special cases.

Binary Addition and Carry

Column Addition

Binary addition proceeds bit by bit, carrying into the next higher column when needed.

```
    1111
+   0001
--------
  1 0000
```

Unsigned Meaning

- In 4 bits, only the lower `0000` remains.
- The extra leading bit is carry-out.
- For unsigned arithmetic, carry-out means the mathematical result exceeded the available range.

Fixed-Width Result

An n-bit register keeps only n result bits. Extra information may be reflected in condition flags.

Unsigned Ranges

Range Formula

An n-bit unsigned integer represents values from 0 to $2^n - 1$.

Width	Smallest	Largest
8 bits	0	255
32 bits	0	$2^{32} - 1$
64 bits	0	$2^{64} - 1$

Teaching Point

Unsigned overflow is not mysterious. The mathematical result is too large for the fixed-width container, so the stored result wraps modulo 2^n.

Two's-Complement Subtraction

Hardware Idea

Subtraction can reuse the same adder by adding the two's-complement negation.

$$A - B = A + (\sim B) + 1$$

AArch64 Example

```
sub x0, x1, x2
```

- Computes `x1 - x2`.
- Writes the fixed-width result to `x0`.
- Does not update NZCV flags.

Important Interpretation

- Negative values are encoded using two's complement.
- The same adder can serve add and subtract.
- Flags are needed to reason about borrow and overflow.

Signed vs. Unsigned Interpretation

Same Bits, Different Values

The same bit pattern can have different numeric meanings depending on whether software treats it as unsigned or signed two's complement.

8-bit pattern	Unsigned	Signed two's complement
`01111111`	127	127
`10000000`	128	-128
`11111111`	255	-1

Connection to AArch64

The instruction result is a bit pattern. The choice of signed or unsigned branch determines how flags are interpreted later.

Signed Overflow

Definition

Signed overflow occurs when the true mathematical result of a two's-complement operation falls outside the signed range representable in the destination width.

Positive Overflow

```
0111 + 0001 = 1000
```

In 4-bit signed arithmetic, $7 + 1$ cannot be represented. The stored pattern looks like -8.

Rule of Thumb

- Add two positives and get a negative: overflow.
- Add two negatives and get a positive: overflow.
- Add opposite signs: no signed overflow.

Carry and Overflow Are Different

Do Not Treat Them as Synonyms

Carry describes unsigned range behavior. Overflow describes signed two's-complement range behavior.

Flag	Main use	Example idea
`C`	unsigned carry or no-borrow	Did unsigned arithmetic exceed range?
`V`	signed overflow	Did signed result exceed two's-complement range?

Course Connection

Later branch instructions use different flag combinations for signed and unsigned comparisons.

Section 02

Condition Flags and Comparisons

AArch64 NZCV Flags

- **N**: result sign bit is 1.
- **Z**: result is exactly zero.
- **C**: carry-out for addition; no-borrow for subtraction.
- **V**: signed two's-complement overflow.

Interpretation Rule

`C` is mainly used for unsigned reasoning; `V` is mainly used for signed overflow reasoning.

Flag-Setting Instructions

No Flag Update

```
add x0, x1, x2
sub x0, x1, x2
```

- Write a register result.
- Leave NZCV unchanged.

Flag Update

```
adds x0, x1, x2
subs x0, x1, x2
```

- Write a register result.
- Update NZCV from that result.

Important Limitation

The `s` suffix is useful here, but do not teach it as a universal rule that applies to every AArch64 mnemonic.

`cmp` as Flag-Setting Subtraction

Comparison Without Keeping the Difference

`cmp` subtracts for flags and discards the numeric result.

```
cmp x0, x1
subs xzr, x0, x1
```

- Both lines express the same underlying comparison mechanism.
- `x0 - x1` updates NZCV.
- The result is written to `xzr`, so it is discarded.

Carry Means No-Borrow After Subtraction

AArch64 Subtraction Rule

After `subs` or `cmp`, `C=1` means the subtraction did **not** require an unsigned borrow.

Operation	Unsigned relation	Carry flag
`5 - 3`	$5 \geq 3$	`C=1`: no borrow
`3 - 5`	$3 < 5$	`C=0`: borrow needed

Common Mistake

Do not read `C=1` after subtraction as "borrow occurred." In AArch64, it means **no borrow**.

Signed and Unsigned Branches

After a Compare

The same `cmp` can support signed or unsigned comparisons. The branch mnemonic determines how the flags are interpreted.

Branch	Interpretation	Condition
`b.lt label`	signed less than	$N \neq V$
`b.ge label`	signed greater/equal	$N = V$
`b.lo label`	unsigned lower	$C = 0$
`b.hs label`	unsigned higher/same	$C = 1$

```
cmp x0, x1   b.lt signed_less   b.lo unsigned_lower
```

Section 03

Multiplication, Division, and Shifts

Multiplication and Division Basics

Architectural Category

Multiply and divide read registers and write register results; latency is implementation-dependent.

Multiply

```
mul x0, x1, x2
```

- Computes a register product.
- May use a distinct multiply unit internally.

Divide

```
udiv x0, x1, x2
sdiv x0, x1, x2
```

- `udiv`: unsigned division.
- `sdiv`: signed division.

Shifts and Powers of Two

Shift	Meaning	Typical interpretation
`lsl #n`	shift left, zeros enter right	multiply by 2^n if overflow is ignored
`lsr #n`	logical right, zeros enter left	unsigned division by powers of two
`asr #n`	arithmetic right, sign bit is replicated	preserves signed negative values
`ror #n`	rotate right, bits wrap around	bit manipulation, not division

Key Distinction

`lsr` and `asr` differ when the top bit is set. That difference matters for unsigned versus signed interpretation.

Shift Example

Logical Shift Right

```
lsr x0, x1, #1
```

- Zero enters the top bit.
- Good for unsigned bit patterns.
- Does not preserve a negative sign.

Arithmetic Shift Right

```
asr x0, x1, #1
```

- Original sign bit is copied.
- Preserves signed interpretation.
- Useful for signed right shifts.

Qualification

Shifts can resemble multiplication or division by powers of two, but finite width, overflow, and signed rounding behavior must be considered.

Section 04

Processor Performance Basics

Execution Time and Performance

Primary Metric

For a fixed workload, performance is the inverse of execution time.

$$\text{Performance} = \frac{1}{\text{Execution Time}}$$

Latency

- Time to complete one task.
- Important for response time.
- Example: seconds per program run.

Throughput

- Work completed per unit time.
- Important for pipelines and servers.
- Example: instructions per cycle.

The Iron Law of Processor Performance

CPU Execution-Time Equation

$$T_{\text{CPU}} = \text{IC} \times \text{CPI} \times T_{\text{clk}} = \frac{\text{IC} \times \text{CPI}}{f_{\text{clk}}}$$

Term	Meaning	Influenced by
IC	dynamic instruction count	algorithm, compiler, ISA
CPI	cycles per instruction	pipeline, memory, branches
T_{clk}	seconds per cycle	circuit timing, clock frequency

Instruction Count Is Dynamic

Dynamic Instruction Count

Dynamic instruction count is the number of instructions actually executed, not merely the number of instructions visible in a source or assembly listing.

- Loops multiply the effect of a small instruction sequence.
- Branches choose which instructions execute.
- Function calls, recursion, and input data can change the count.
- Optimizing one program region may not matter if it rarely executes.

Simple Example

A three-instruction loop body executed N times contributes approximately $3N$ dynamic instructions, before considering branch and loop-control instructions.

CPI and IPC

Cycles Per Instruction

$$\text{CPI} = \frac{\text{cycles}}{\text{instructions}}$$

- Average over a workload.
- Can include stalls and cache misses.
- Lower is usually better.

Instructions Per Cycle

$$\text{IPC} = \frac{1}{\text{CPI}}$$

- Average completed instructions per cycle.
- Can exceed 1 on superscalar cores.
- Higher is usually better.

No Universal Cycle Count

Instruction latency and throughput are microarchitecture-dependent, not fixed by the AArch64 ISA alone.

Average CPI from an Instruction Mix

Weighted Average

When instruction classes have different average costs, overall CPI depends on the dynamic instruction mix.

$$\text{Average CPI} = \sum_i (\text{fraction}_i \times \text{CPI}_i)$$

Class	Fraction	CPI	Contribution
A	50%	1	0.50
B	30%	2	0.60
C	20%	4	0.80

$$\text{Average CPI} = 1.90$$

Clock Frequency Is Not Performance

Common Mistake

A higher clock rate does not automatically mean a shorter runtime. Instruction count and CPI matter just as much.

Processor	Clock	CPI	Time / instruction
A	4.0 GHz	2.0	0.50 ns
B	3.0 GHz	1.0	0.333 ns

Interpretation

Processor B has a lower frequency, but each instruction takes fewer cycles on average, so its average time per instruction is lower for this workload.

Worked Performance Example

Machine A

- IC $= 1.0 \times 10^9$
- CPI $= 2.0$
- $f = 4.0$ GHz

$$T_A = \frac{1.0 \times 10^9 \times 2.0}{4.0 \times 10^9} = 0.50 \text{ s}$$

Machine B

- IC $= 1.2 \times 10^9$
- CPI $= 0.8$
- $f = 2.5$ GHz

$$T_B = \frac{1.2 \times 10^9 \times 0.8}{2.5 \times 10^9} = 0.384 \text{ s}$$

$$\text{Speedup} = \frac{0.50}{0.384} \approx 1.30\times$$

Section 05

Summary and Self-Test

Topic 4 Takeaways

- Integer arithmetic operates on fixed-width bit patterns.
- Signedness comes from interpretation, not from the adder itself.
- `C` supports unsigned reasoning; `V` supports signed overflow reasoning.
- After subtraction, `C=1` means no unsigned borrow occurred.
- Shifts can support power-of-two scaling, but overflow and signed behavior matter.
- Execution time depends jointly on instruction count, CPI, and clock period.

Self-Test 1/2

1. Why do add and sub not by themselves determine signedness?
2. What does an unsigned carry-out mean in fixed-width addition?
3. How can subtraction be implemented using addition and two's complement?
4. What is the difference between `C` and `V`?
5. After `cmp x0, x1`, what does `C=1` mean for unsigned comparison?

Self-Test 2/2

6. What is the difference between `add` and `adds`?
7. Explain why `cmp x0, x1` can be understood as `subs xzr, x0, x1`.
8. Why is `asr` different from `lsr` when the sign bit is set?
9. What three terms appear in the CPU execution-time equation?
10. Why can a lower-frequency processor outperform a higher-frequency processor?

Wrap-Up and Next Steps

Next Topic: AArch64 Arithmetic, Logic, Shifts, and Flags

The next lecture moves from arithmetic foundations to concrete AArch64 instruction forms and bit-level manipulation patterns.

- Arithmetic and logical instruction families
- Aliases such as `cmp` and `tst`
- Logical masks and shifts
- Immediate operands and assembler conveniences